Лабораторная работа №2

Первоначальна настройка git

Диденко Герман Максимович

Содержание

1 Цель работы **5**

2 Задание **6**

3 Теоретическое введение **7**

4 Выполнение лабораторной работы **8**

4.1 Настройка git и gh 8

5 Контрольные вопросы **12**

6 Выводы **14**

7 Список литературы **15**

Список иллюстраций

4.1 Установка git и gh 8
4.2 Настройки git 8
4.3 Ключ с rsa алгоритмом 9
4.4 gpg –full-generate-key 9
4.5 PGP Fingerprint 9
4.6 Создание репозитория 10
4.7 Доп. настройки git 10
4.8 Авторизация github 10
4.9 Создание репозитория 11
4.10 Загрузка каталогов 11

Список таблиц

1 Цель работы

Целью данной работы является изучить идеологию и применение средств контроля версий, освоить умения по работе с git.

---

2 Задание

- Установка и настройка git.
- Ключи для авторизации
- Каталог для выполнения заданий по предмету

---

3 Теоретическое введение

Системы контроля версий (Version Control System, VCS) применяются при работе нескольких человек над одним проектом. Обычно основное дерево проекта хранится в локальном или удалённом репозитории, к которому настроен доступ для участников проекта. При внесении изменений в содержание проекта система контроля версий позволяет их фиксировать, совмещать изменения, произведённые разными участниками проекта, производить откат к любой более ранней версии проекта, если это требуется.

---

4 Выполнение лабораторной работы

4.1 Настройка git и gh

Устанавливаю git и gh через root

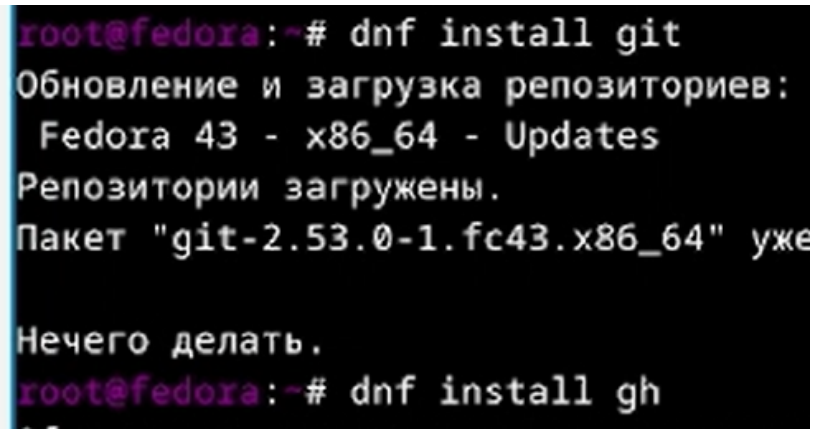

Рисунок 4.1: Установка git и gh

Делаю настройки для git

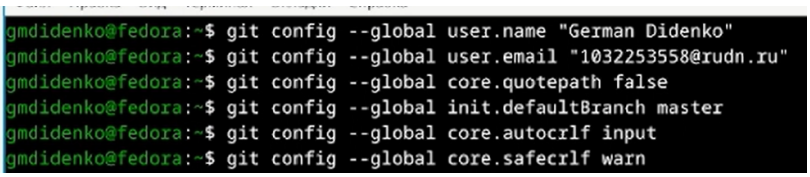

Рисунок 4.2: Настройки git

Создаю ключ для авторизации

```
Your public key has been saved in /home/gmdidenko/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:PKT+yFa+M5x3HEchs6ZcLExHiCBP5vM1dtKXsfwSFQM gmdidenko@fedora
The key's randomart image is:
+---[RSA 4096]----+
|     . +. . o.Eooo|
|      *  . o.+..=.|
|       + .o=oo+*. |
|        * oo+=..o |
|       . S. = .. .|
|      .  ..o . .. |
|       .+ . . o   |
|      ..o* . o    |
|      .o o= .     |
+----[SHA256]-----+
```

Рисунок 4.3: Ключ с rsa алгоритмом

Также создаю PGP ключ

```
gmdidenko@fedora:~$ gpg --full-generate-key
gpg (GnuPG) 2.4.8; Copyright (C) 2025 g10 Code GmbH
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

gpg: создан каталог '/home/gmdidenko/.gnupg'
```

Рисунок 4.4: gpg –full-generate-key

Вывожу fingerprint

```
gmdidenko@fedora:~$ gpg --list-secret-keys --keyid-format LONG
gpg: проверка таблицы доверия
gpg: marginals needed: 3  completes needed: 1  trust model: pgp
gpg: глубина: 0  достоверных:   1  подписанных:   0  доверие: 0-, 0q, 0n, 0m, 0f, 1u
[keyboxd]
---------
sec   rsa4096/F7C133CB6C99E3FD 2026-03-06 [SC]
```

Рисунок 4.5: PGP Fingerprint

Ввожу ключ на Github

Рисунок 4.6: Создание репозитория

Делаю настройку для git

```
gmdidenko@fedora:~$ git config --global user.signingkey F7C133CB6C99E3FD
gmdidenko@fedora:~$ git config --global commit.gpgsign true
gmdidenko@fedora:~$ git config --global gpg.program $(which gpg2)
```

Рисунок 4.7: Доп. настройки git

С помощью `gh auth login` авторизовываюсь на github

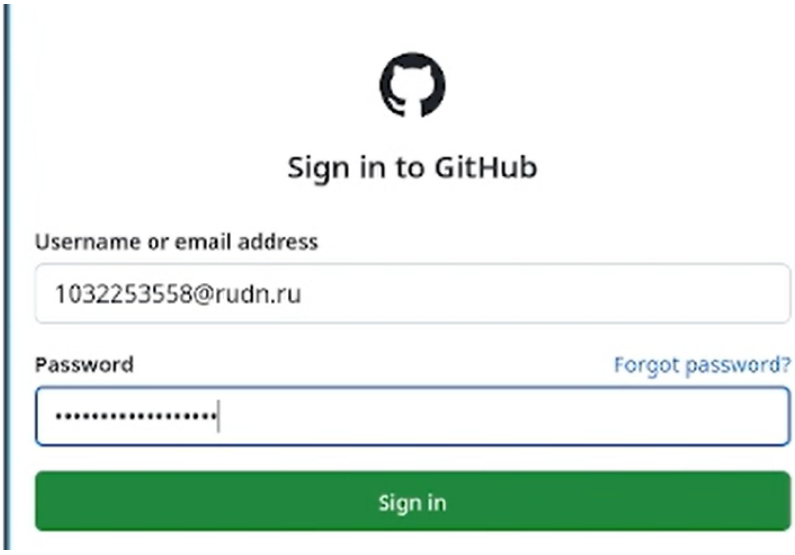

Рисунок 4.8: Авторизация github

Создаю репозиторий на своем github

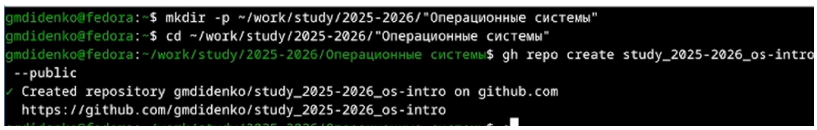

Рисунок 4.9: Создание репозитория

Создаю каталоги в репозитории и загружаю их на Github

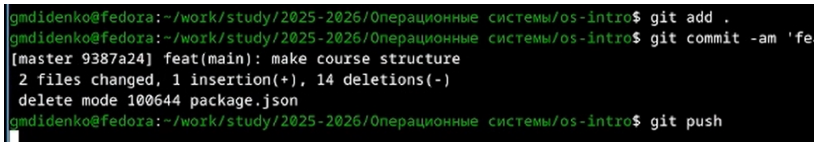

Рисунок 4.10: Загрузка каталогов

---

5 Контрольные вопросы

1. Что такое система контроля версий?

Инструмент для фиксации изменений в файлах, позволяющий отслеживать историю, возвращаться к предыдущим версиям и совмещать правки разных авторов.

2. Централизованные и распределённые VCS: в чём разница?

Централизованные (SVN) хранят историю на одном сервере; распределённые (Git) дают каждому участнику полную копию репозитория, позволяя работать офлайн.

3. Для чего нужны `git add` и `git commit`?

`git add` — добавляет изменения в индекс (подготовка к коммиту); `git commit` — сохраняет изменения в локальную историю с комментарием.

4. Зачем нужны SSH-ключи в Git?

Для безопасной аутентификации при подключении к удалённым репозиториям без ввода логина и пароля.

5. Чем отличается PGP-подпись коммита?

Криптографически подтверждает авторство и целостность коммита; на GitHub такие коммиты помечаются как **Verified**.

6. Что делает `git config --global`?

Записывает настройки в глобальный файл `~/.gitconfig`, применяя их ко всем репозиториям пользователя.

7. Зачем нужен `core.autocrlf` и какое значение в Linux?

Управляет преобразованием символов конца строки. В Linux: `input` — конвер-

тирует CRLF→LF при коммите, не меняет при checkout.

8. Как проверить подпись коммита?

Локально: `git log --show-signature`; на GitHub — значок **Verified** рядом с коммитом.

9. Что делает `git flow init`?

Инициализирует структуру веток Git Flow: создаёт `master`, `develop` и настраивает префиксы для `feature/`, `release/`, `hotfix/`.

10. Как создать и отправить тег версии?

```
git tag -a v1.0.0 -m "Release"
git push origin v1.0.0
# или все теги:
git push --tags
```

6 Выводы

В ходе выполнения лабораторной работы были приобретены следующие навыки работы с git, gh.

---

7 Список литературы

1) Официальная документация Git. — URL: https://git-scm.com/doc (дата обращения: 2026)
2) GitHub Docs. — URL: https://docs.github.com (дата обращения: 2026)
3) Проктор Г. Git для профессионалов. — М.: ДМК Пресс, 2021. — 352 с.
4) Чакона С., Штрауб Б. Pro Git. 2-е изд. — М.: Питер, 2020. — 416 с.
5) Семантическое версионирование. — URL: https://semver.org/lang/ru/ (дата обращения: 2026)
6) Conventional Commits. — URL: https://www.conventionalcommits.org (дата обращения: 2026)
7) Pandoc User's Guide. — URL: https://pandoc.org/MANUAL.html (дата обращения: 2026)
8) ГОСТ 7.32-2001. Отчёт о научно-исследовательской работе. Структура и правила оформления.